<u>CSE</u> Department – Faculty of Engineering - MSA

CSE462a/CSE242 Fundamentals of Database Systems – Spring 2026

Project Report

Fundamentals of Database Systems (CSE462a)

Name:	**ID:**
Adham Hassan	242335
Yaseen Mohamed	247699
Kareem ahmed	247225

Supervised by: Dr. Ahmed Ayoub

Abstract

This document presents a detailed report on the project titled Public Transport Management System. The project is designed to organize the main operations of public transportation services such as routes, stations, schedules, passengers, ticket booking, and payments. The report covers the problem statement, objectives, database design, implementation details, query optimization, and conclusions.

Contents

1 Introduction **3**
- 1.1 Problem Statement 3
- 1.2 Objectives 3
- 1.3 Scope 3

2 Database Design **4**
- 2.1 Entity-Relationship Diagram (ERD) 4
- 2.2 Normalization 5
- 2.3 Schema Design 7

3 Implementation **8**
- 3.1 Database Tables 8
- 3.2 SQL Queries 10
- 3.3 Stored Procedures, Triggers, Transactions 12

4 Query Optimization **13**

5 Data Integrity and Security **14**
- 5.1 Constraints and Data Validation 14
- 5.2 Access Control and Security Measures 15

6 Results and Discussion **16**
- 6.1 Testing and Validation 16
- 6.2 Challenges and Limitations 16

8 References

1 Introduction

1.1 Problem Statement

Public transportation systems, such as buses, metro lines, and trains, deal with a large amount of daily information. This includes passenger data, vehicle details, stations, routes, trip schedules, ticket bookings, and payment records. If this data is not stored in an organized way, several problems may appear, such as duplicated passenger records, incorrect ticket information, wrong seat availability, or difficulty in finding schedules.
The main problem this project aims to solve is how to design a structured database that can manage the basic operations of a public transport company. Instead of storing the data manually or in separate files, the system stores related information in connected tables. This makes the data easier to search, update, validate, and protect.

1.2 Objectives

The main objectives of this project are:

- To design a database for a Public Transport Management System.
- To store passenger information in a clear and organized way.
- To manage user accounts for passengers and administrators.
- To store station information for different transport types such as bus, metro, and train.
- To manage vehicle data, including vehicle type, number, capacity, and status.
- To define routes between stations.
- To store route stops in the correct order.
- To create schedules for routes and assigned vehicles.
- To allow passengers to book tickets for available schedules.
- To store payment records for booked tickets.
- To use transactions to make the booking process more reliable.
- To use indexing to improve schedule search performance.
- To implement a stored procedure for fare calculation.

1.3 Scope

The scope of this project is focused on the database part of a public transport system. The project includes database design, ERD design, normalization, table creation, sample data insertion, SQL queries, stored procedures, transactions, indexing, and basic data integrity rules.
The system does not include a full website or mobile application. It also does not include real-time GPS tracking, online payment gateway integration, or live traffic up-dates. These features can be added later as future improvements. The current version is mainly concerned with building a correct and well-structured relational database.

2 Database Design

2.1 Entity-Relationship Diagram (ERD)

The ERD represents the main entities in the Public Transport Management System and the relationships between them. The main entities are Passenger, User Account, Station, Vehicle, Route, Route Stop, Schedule, Ticket, and Payment.
The diagram uses crow's foot notation to show the cardinality between entities. For example, one passenger can book many tickets, but each ticket belongs to one passenger. Also, one route can have many schedules, and each schedule belongs to one route.

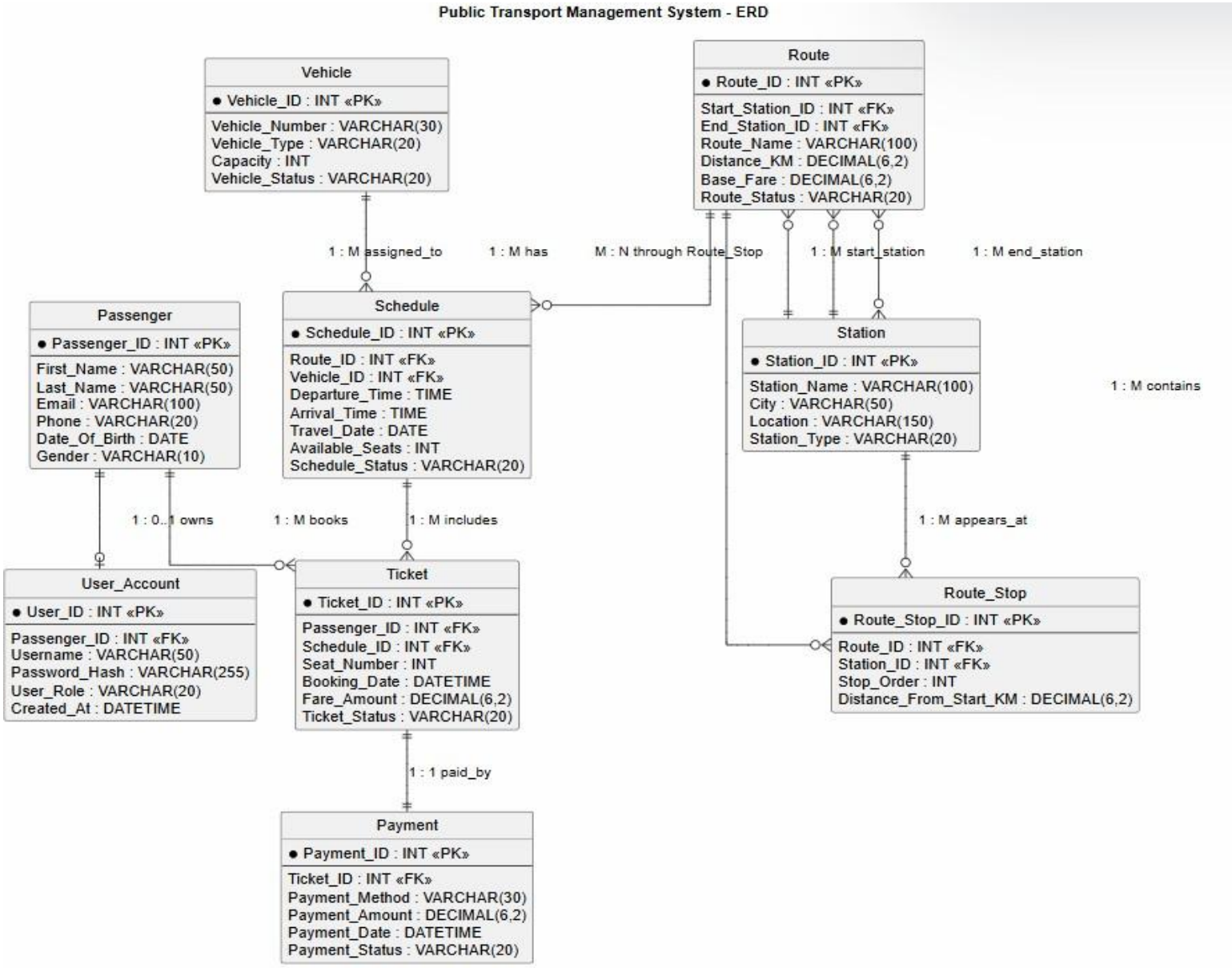

Figure 1: Entity-Relationship Diagram for Public Transport Management System

The relationships in the ERD are as follows.

Relationship	Cardinality	Description
Passenger to User Account	1 to 0..1	One passenger may have zero or one user account. This is useful because an administrator account may not be linked to a passenger.
Passenger to Ticket	1 to many	One passenger can book many tickets, while each ticket belongs to one passenger.
Station to Route as start station	1 to many	One station can be the starting station for many routes.
Station to Route as end station	1 to many	One station can be the ending station for many routes.
Route to Route_Stop	1 to many	One route can contain many stops.
Station to Route_Stop	1 to many	One station can appear in many route stops.
Route to Schedule	1 to many	One route can have many schedules.
Vehicle to Schedule	1 to many	One vehicle can be assigned to many schedules.
Schedule to Ticket	1 to many	One schedule can include many booked tickets.
Ticket to Payment	1 to 0..1	One ticket may have zero or one payment record.

The many-to-many relationship between Route and Station is resolved using the Route Stop associative entity. A route may pass through many stations, and a sta-tion may appear in many routes. Instead of storing this directly, Route Stop stores the Route ID, Station ID, Stop Order, and Distance from Start KM.

2.2 Normalization

The database was designed to follow Third Normal Form, also known as 3NF. Normalization was applied to reduce duplicated data and avoid update, insertion, and deletion problems.

First Normal Form

The database satisfies First Normal Form because each table contains atomic values. For example, the Passenger table stores First_Name and Last_Name separately, and each field contains only one value. There are no repeating groups inside a single column.

Second Normal Form

The database satisfies Second Normal Form because every non-key attribute depends on the primary key of its table. Since the tables use single-column primary keys, there are no partial dependencies. For example, in the Ticket table, the seat number, booking date, fare amount, and ticket status all depend on Ticket ID.

Third Normal Form

The database satisfies Third Normal Form because non-key attributes do not depend on other non-key attributes. Passenger details are stored only in the Passenger table, vehicle details are stored only in the Vehicle table, and station details are stored only in the Station table. The Ticket table does not repeat passenger name, route name, or vehicle details. It only stores foreign keys that connect it to the related tables.

This design helps keep the data consistent. For example, if a passenger phone number changes, it only needs to be updated in the Passenger table instead of being changed in many ticket records.

Denormalization

The design does not use major denormalization. Most data is stored in separate related tables to avoid redundancy. However, the Fare Amount attribute is stored in the Ticket table even though fare can be calculated from route distance and base fare. This was done intentionally because the fare paid at booking time should be saved as a historical value. If the route fare changes later, old tickets should still show the actual fare that was paid at the time of booking.

2.3 Schema Design

The database schema contains nine main tables. Each table has a primary key, and related tables are connected using foreign keys.

Table	Primary Key	Purpose
Passenger	Passenger_ID	Stores passenger personal information such as name, email, phone, date of birth, and gender.
User_Account	User_ID	Stores login information, password hash, user role, and account creation date.
Station	Station_ID	Stores station information such as station name, city, location, and type.
Vehicle	Vehicle_ID	Stores vehicle information such as vehicle number, type, capacity, and status.
Route	Route_ID	Stores route information including start station, end station, distance, base fare, and status.
Route_Stop	Route_Stop_ID	Stores intermediate stops for each route and the order of these stops.
Schedule	Schedule_ID	Stores trip schedules, including route, vehicle, departure time, arrival time, travel date, and available seats.
Ticket	Ticket_ID	Stores ticket booking details for passengers.
Payment	Payment_ID	Stores payment information for tickets.

The following relational scheme summarizes the database:

Passenger(Passenger_ID, First_Name, Last_Name, Email, Phone,
Date_Of_Birth, Gender)

User_Account(User_ID, Passenger_ID, Username, Password_Hash,
User_Role, Created_At)

Station(Station_ID, Station_Name, City, Location, Station_Type)

Vehicle(Vehicle_ID, Vehicle_Number, Vehicle_Type, Capacity, Vehicle_Status)

Route(Route_ID, Route_Name, Start_Station_ID, End_Station_ID, Distance_KM, Base_Fare, Route_Status)

Route_Stop(Route_Stop_ID, Route_ID, Station_ID, Stop_Order, Distance_From_Start_KM)

Schedule(Schedule_ID, Route_ID, Vehicle_ID, Departure_Time, Arrival_Time, Travel_Date, Available_Seats, Schedule_Status)

Ticket(Ticket_ID, Passenger_ID, Schedule_ID, Seat_Number, Booking_Date, Fare_Amount, Ticket_Status)

Payment(Payment_ID, Ticket_ID, Payment_Method, Payment_Amount, Payment_Date, Payment_Status)

A small correction is recommended for the Route foreign keys. The original SQL repeated the foreign key for End_Station_ID twice. The corrected version should include both Start_Station_ID and End_Station_ID as foreign keys.

```
ALTER TABLE Route
ADD FOREIGN KEY (Start_Station_ID)
REFERENCES Station(Station_ID);

ALTER TABLE Route
ADD FOREIGN KEY (End_Station_ID)
REFERENCES Station(Station_ID);
```

3 Implementation

3.1 Database Tables

The database was implemented using MySQL. The database name is publictransportdb. The implementation includes table creation, primary keys, foreign keys, sample data, stored procedure, index, queries, and transactions.

The main tables are described below:

Table	Main Attributes and Data Types
Passenger	Passenger_ID INT, First_Name VARCHAR(50), Last_Name VARCHAR(50), Email VARCHAR(100), Phone VARCHAR(20), Date_Of_Birth DATE, Gender VARCHAR(10)
User_Account	User_ID INT, Passenger_ID INT, Username VARCHAR(50), Password_Hash VARCHAR(255), User_Role VARCHAR(20), Created_At DATETIME

Station	Station_ID INT, Station_Name VARCHAR(100), City VARCHAR(50), Location VARCHAR(150), Station_Type VARCHAR(20)
Vehicle	Vehicle_ID INT, Vehicle_Number VARCHAR(30), Vehicle_Type VARCHAR(20), Capacity INT, Vehicle_Status VARCHAR(20)
Route	Route_ID INT, Route_Name VARCHAR(100), Start_Station_ID INT, End_Station_ID INT, Distance_KM DECIMAL(6,2), Base_Fare DECIMAL(6,2), Route_Status VARCHAR(20)
Route_Stop	Route_Stop_ID INT, Route_ID INT, Station_ID INT, Stop_Order INT, Distance_From_Start_KM DECIMAL(6,2)
Schedule	Schedule_ID INT, Route_ID INT, Vehicle_ID INT, Departure_Time TIME, Arrival_Time TIME, Travel_Date DATE, Available_Seats INT, Schedule_Status VARCHAR(20)
Ticket	Ticket_ID INT, Passenger_ID INT, Schedule_ID INT, Seat_Number INT, Booking_Date DATETIME, Fare_Amount DECIMAL(6,2), Ticket_Status VARCHAR(20)
Payment	Payment_ID INT, Ticket_ID INT, Payment_Method VARCHAR(30), Payment_Amount DECIMAL(6,2), Payment_Date DATETIME, Payment_Status VARCHAR(20)

Example table creation for the Passenger table:

```
CREATE TABLE Passenger (
    Passenger_ID INT NOT NULL,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
    Email VARCHAR(100),
    Phone VARCHAR(20),
    Date_Of_Birth DATE,
    Gender VARCHAR(10),
    PRIMARY KEY (Passenger_ID)
);
```

Example table creation for the Ticket table:

```
CREATE TABLE Ticket (
    Ticket_ID INT NOT NULL,
    Passenger_ID INT,
    Schedule_ID INT,
    Seat_Number INT,
    Booking_Date DATETIME,
    Fare_Amount DECIMAL(6,2),
    Ticket_Status VARCHAR(20),
    PRIMARY KEY (Ticket_ID)
);
```

Figure 2: Database Tables Created in MySQL

Passenger_ID	First_Name	Last_Name	Email	Phone	Date_Of_Birth	Gender
1	Ahmed	Ali	ahmed.ali@email.com	01011111111	2002-05-14	Male
2	Sara	Mohamed	sara.mohamed@email.com	01022222222	2003-03-20	Female
3	Omar	Hassan	omar.hassan@email.com	01033333333	2001-11-10	Male
4	Mona	Ibrahim	mona.ibrahim@email.com	01044444444	2002-07-25	Female
5	Youssef	Khaled	youssef.khaled@email.com	01055555555	2000-09-12	Male
6	Nour	Adel	nour.adel@email.com	01066666666	2004-01-30	Female
7	Kareem	Samir	kareem.samir@email.com	01077777777	2001-06-18	Male
8	Laila	Mostafa	laila.mostafa@email.com	01088888888	2003-12-05	Female
9	Hana	Tarek	hana.tarek@email.com	01099999999	2002-10-09	Female
10	Mahmoud	Fathy	mahmoud.fathy@email.com	01111111111	2000-04-22	Male
11	Ali	Nasser	ali.nasser@email.com	01122222222	2001-08-13	Male
12	Farah	Magdy	farah.magdy@email.com	01133333333	2003-02-17	Female
13	Hossam	Reda	hossam.reda@email.com	01144444444	2002-06-01	Male
14	Dina	Ashraf	dina.ashraf@email.com	01155555555	2004-09-19	Female
15	Tamer	Said	tamer.said@email.com	01166666666	2001-03-29	Male
16	Reem	Wael	reem.wael@email.com	01177777777	2002-12-11	Female
17	Mostafa	Gamal	mostafa.gamal@email.com	01188888888	2000-05-07	Male
18	Salma	Sherif	salma.sherif@email.com	01199999999	2003-07-14	Female
19	Hany	Ezz	hany.ezz@email.com	01211111111	2002-01-21	Male
20	Nada	Ramy	nada.ramy@email.com	01222222222	2004-11-03	Female
21	Mariam	Sameh	mariam.sameh@email.com	01233333333	2002-08-08	Female
22	Adham	Nabil	adham.nabil@email.com	01244444444	2001-10-16	Male
23	Jana	Ayman	jana.ayman@email.com	01255555555	2003-04-04	Female
24	Seif	Osama	seif.osama@email.com	01266666666	2000-12-23	Male
25	Malak	Hesham	malak.hesham@email.com	01277777777	2004-06-15	Female
26	Amr	Salah	amr.salah@email.com	01288888888	2001-02-02	Male
27	Yara	Kamal	yara.kamal@email.com	01299999999	2003-09-09	Female
28	Fady	Waheed	fady.waheed@email.com	01511111111	2000-03-13	Male
29	Rana	Essam	rana.essam@email.com	01522222222	2002-07-07	Female
30	Karim	Atef	karim.atef@email.com	01533333333	2001-05-19	Male
NULL	NULL	NULL	NULL	NULL	NULL	NULL

2.1 Passenger

Payment_ID	Ticket_ID	Payment_Method	Payment_Amount	Payment_Date	Payment_Status
1	1	Cash	17.75	2026-04-25 09:01:00	Paid
2	2	Card	14.00	2026-04-25 09:06:00	Paid
3	3	Cash	12.50	2026-04-25 09:11:00	Paid
4	4	Mobile Wallet	15.00	2026-04-25 09:16:00	Paid
5	5	Cash	9.00	2026-04-25 09:21:00	Paid
6	6	Card	22.00	2026-04-25 09:26:00	Paid
7	7	Mobile Wallet	18.00	2026-04-25 09:31:00	Paid
8	8	Card	130.00	2026-04-25 09:36:00	Paid
9	9	Cash	62.50	2026-04-25 09:41:00	Paid
10	10	Card	77.50	2026-04-25 09:46:00	Paid
11	11	Cash	57.50	2026-04-25 09:51:00	Paid
12	12	Mobile Wallet	75.00	2026-04-25 09:56:00	Paid
13	13	Card	82.50	2026-04-25 10:01:00	Paid
14	14	Cash	65.00	2026-04-25 10:06:00	Paid
15	15	Mobile Wallet	72.50	2026-04-25 10:11:00	Paid
16	16	Card	142.50	2026-04-25 10:16:00	Paid
17	17	Cash	207.50	2026-04-25 10:21:00	Paid
18	18	Card	255.00	2026-04-25 10:26:00	Paid
19	19	Mobile Wallet	360.00	2026-04-25 10:31:00	Paid
20	20	Cash	70.00	2026-04-25 10:36:00	Paid
21	21	Card	17.75	2026-04-25 10:41:00	Paid
22	22	Cash	130.00	2026-04-25 10:46:00	Paid
23	23	Mobile Wallet	62.50	2026-04-25 10:51:00	Paid
24	24	Card	75.00	2026-04-25 10:56:00	Paid
25	25	Cash	360.00	2026-04-25 11:01:00	Paid
26	26	Card	17.75	2026-04-25 11:06:00	Paid
27	27	Mobile Wallet	14.00	2026-04-25 11:11:00	Paid
28	28	Cash	130.00	2026-04-25 11:16:00	Paid
29	29	Card	360.00	2026-04-25 11:21:00	Paid
30	30	Mobile Wallet	360.00	2026-04-25 11:26:00	Paid
NULL	NULL	NULL	NULL	NULL	NULL

2.2 Payment

Route_ID	Route_Name	Start_Station_ID	End_Station_ID	Distance_KM	Base_Fare	Route_Status
1	Cairo to Giza	1	2	25.50	5.00	Active
2	Cairo to Nasr City	1	3	18.00	5.00	Active
3	Cairo to Maadi	1	4	15.00	5.00	Active
4	Maadi to Helwan	4	5	20.00	5.00	Active
5	Giza to Dokki	2	6	8.00	5.00	Active
6	Giza to October	2	7	30.00	7.00	Active
7	Nasr City to New Cairo	3	8	22.00	7.00	Active
8	Cairo to Alexandria	1	9	220.00	20.00	Active
9	Cairo to Tanta	1	10	95.00	15.00	Active
10	Cairo to Mansoura	1	11	125.00	15.00	Active
11	Cairo to Zagazig	1	12	85.00	15.00	Active
12	Cairo to Ismailia	1	13	120.00	15.00	Active
13	Cairo to Suez	1	14	135.00	15.00	Active
14	Cairo to Fayoum	1	15	100.00	15.00	Active
15	Cairo to Beni Suef	1	16	115.00	15.00	Active
16	Cairo to Minya	1	17	245.00	20.00	Active
17	Cairo to Asyut	1	18	375.00	20.00	Active
18	Cairo to Sohag	1	19	470.00	20.00	Active
19	Cairo to Luxor	1	20	670.00	25.00	Active
20	Alexandria to Tanta	9	10	110.00	15.00	Active
NULL	NULL	NULL	NULL	NULL	NULL	NULL

2.3 Route

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Schedule_ID	Route_ID	Vehicle_ID	Departure_Time	Arrival_Time	Travel_Date	Available_Seats	Schedule_Status
1	1	1	08:00:00	09:00:00	2026-05-01	48	Scheduled
2	2	2	09:00:00	10:00:00	2026-05-01	43	Scheduled
3	3	6	10:00:00	10:45:00	2026-05-01	219	Scheduled
4	4	7	11:00:00	12:00:00	2026-05-01	219	Scheduled
5	5	8	12:00:00	12:30:00	2026-05-01	209	Scheduled
6	6	3	13:00:00	14:30:00	2026-05-01	39	Scheduled
7	7	4	14:00:00	15:15:00	2026-05-01	49	Scheduled
8	8	9	07:00:00	10:30:00	2026-05-02	298	Scheduled
9	9	10	08:00:00	09:45:00	2026-05-02	319	Scheduled
10	10	11	09:00:00	11:15:00	2026-05-02	309	Scheduled
11	11	12	10:00:00	11:30:00	2026-05-02	349	Scheduled
12	12	14	11:00:00	13:00:00	2026-05-02	49	Scheduled
13	13	15	12:00:00	14:15:00	2026-05-02	39	Scheduled
14	14	5	13:00:00	15:00:00	2026-05-02	54	Scheduled
15	15	18	14:00:00	16:20:00	2026-05-02	329	Scheduled
16	16	19	15:00:00	19:00:00	2026-05-03	339	Scheduled
17	17	20	16:00:00	22:00:00	2026-05-03	359	Scheduled
18	18	9	17:00:00	23:30:00	2026-05-03	299	Scheduled
19	19	10	18:00:00	05:00:00	2026-05-03	318	Scheduled
20	20	11	19:00:00	21:00:00	2026-05-03	309	Scheduled
21	1	2	16:00:00	17:00:00	2026-05-04	44	Scheduled
22	8	12	06:30:00	10:00:00	2026-05-04	349	Scheduled
23	9	18	07:30:00	09:15:00	2026-05-04	329	Scheduled
24	12	19	08:30:00	10:45:00	2026-05-04	339	Scheduled
25	19	20	20:00:00	07:00:00	2026-05-04	358	Scheduled
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2.4Schedule

Station_ID	Station_Name	City	Location	Station_Type
1	Cairo Central Station	Cairo	Ramses Square	Train
2	Giza Station	Giza	Giza Square	Train
3	Nasr City Station	Cairo	Nasr City	Bus
4	Maadi Station	Cairo	Maadi	Metro
5	Helwan Station	Cairo	Helwan	Metro
6	Dokki Station	Giza	Dokki	Metro
7	October Station	Giza	6th October	Bus
8	New Cairo Station	Cairo	New Cairo	Bus
9	Alexandria Station	Alexandria	Misr Station	Train
10	Tanta Station	Gharbia	Tanta City	Train
11	Mansoura Station	Dakahlia	Mansoura City	Train
12	Zagazig Station	Sharqia	Zagazig City	Train
13	Ismailia Station	Ismailia	Ismailia City	Train
14	Suez Station	Suez	Suez City	Bus
15	Fayoum Station	Fayoum	Fayoum City	Bus
16	Beni Suef Station	Beni Suef	Beni Suef City	Train
17	Minya Station	Minya	Minya City	Train
18	Asyut Station	Asyut	Asyut City	Train
19	Sohag Station	Sohag	Sohag City	Train
20	Luxor Station	Luxor	Luxor City	Train
21	Abbasiya Station	Cairo	Abbasiya	Bus
22	Tora Station	Cairo	Tora	Metro
23	Sadat Station	Cairo	Tahrir Square	Metro
24	Damanhur Station	Beheira	Damanhur City	Train
25	Qalyub Station	Qalyubia	Qalyub City	Train
NULL	NULL	NULL	NULL	NULL

2.5 Station

2.6 Ticket

Ticket_ID	Passenger_ID	Schedule_ID	Seat_Number	Booking_Date	Fare_Amount	Ticket_Status
1	1	1	27	2026-04-25 09:00:00	17.75	Booked
2	2	2	14	2026-04-25 09:05:00	14.00	Booked
3	3	3	88	2026-04-25 09:10:00	12.50	Booked
4	4	4	156	2026-04-25 09:15:00	15.00	Booked
5	5	5	42	2026-04-25 09:20:00	9.00	Booked
6	6	6	31	2026-04-25 09:25:00	22.00	Booked
7	7	7	9	2026-04-25 09:30:00	18.00	Booked
8	8	8	214	2026-04-25 09:35:00	130.00	Booked
9	9	9	178	2026-04-25 09:40:00	62.50	Booked
10	10	10	299	2026-04-25 09:45:00	77.50	Booked
11	11	11	73	2026-04-25 09:50:00	57.50	Booked
12	12	12	38	2026-04-25 09:55:00	75.00	Booked
13	13	13	22	2026-04-25 10:00:00	82.50	Booked
14	14	14	49	2026-04-25 10:05:00	65.00	Booked
15	15	15	187	2026-04-25 10:10:00	72.50	Booked
16	16	16	301	2026-04-25 10:15:00	142.50	Booked
17	17	17	345	2026-04-25 10:20:00	207.50	Booked
18	18	18	266	2026-04-25 10:25:00	255.00	Booked
19	19	19	115	2026-04-25 10:30:00	360.00	Booked
20	20	20	287	2026-04-25 10:35:00	70.00	Booked
21	21	21	33	2026-04-25 10:40:00	17.75	Booked
22	22	22	120	2026-04-25 10:45:00	130.00	Booked
23	23	23	210	2026-04-25 10:50:00	62.50	Booked
24	24	24	240	2026-04-25 10:55:00	75.00	Booked
25	25	25	346	2026-04-25 11:00:00	360.00	Booked
26	26	1	44	2026-04-25 11:05:00	17.75	Booked
27	27	2	22	2026-04-25 11:10:00	14.00	Booked
28	28	8	108	2026-04-25 11:15:00	130.00	Booked
29	29	19	215	2026-04-25 11:20:00	360.00	Booked
30	30	25	25	2026-04-25 11:25:00	360.00	Booked
NULL	NULL	NULL	NULL	NULL	NULL	NULL

2.7 User_account

User_ID	Passenger_ID	Username	Password_Hash	User_Role	Created_At
1	1	ahmedali	pass123	Regular_User	2026-04-01 10:00:00
2	2	saramohamed	pass123	Regular_User	2026-04-01 10:05:00
3	3	omarhassan	pass123	Regular_User	2026-04-01 10:10:00
4	4	monaibrahim	pass123	Regular_User	2026-04-01 10:15:00
5	5	youssefkhaled	pass123	Regular_User	2026-04-01 10:20:00
6	6	nouradel	pass123	Regular_User	2026-04-01 10:25:00
7	7	kareemsamir	pass123	Regular_User	2026-04-01 10:30:00
8	8	lailamostafa	pass123	Regular_User	2026-04-01 10:35:00
9	9	hanatarek	pass123	Regular_User	2026-04-01 10:40:00
10	10	mahmoudfathy	pass123	Regular_User	2026-04-01 10:45:00
11	11	alinasser	pass123	Regular_User	2026-04-01 10:50:00
12	12	farahmagdy	pass123	Regular_User	2026-04-01 10:55:00
13	13	hossamreda	pass123	Regular_User	2026-04-01 11:00:00
14	14	dinaashraf	pass123	Regular_User	2026-04-01 11:05:00
15	15	tamersaid	pass123	Regular_User	2026-04-01 11:10:00
16	16	reemwael	pass123	Regular_User	2026-04-01 11:15:00
17	17	mostafagamal	pass123	Regular_User	2026-04-01 11:20:00
18	18	salmasherif	pass123	Regular_User	2026-04-01 11:25:00
19	19	hanyezz	pass123	Regular_User	2026-04-01 11:30:00
20	20	nadaramy	pass123	Regular_User	2026-04-01 11:35:00
21	21	mariamsameh	pass123	Regular_User	2026-04-01 11:40:00
22	22	adhamnabil	pass123	Regular_User	2026-04-01 11:45:00
23	23	janaayman	pass123	Regular_User	2026-04-01 11:50:00
24	24	seifosama	pass123	Regular_User	2026-04-01 11:55:00
25	25	malakhesham	pass123	Regular_User	2026-04-01 12:00:00
26	26	amrsalah	pass123	Regular_User	2026-04-01 12:05:00
27	27	yarakamal	pass123	Regular_User	2026-04-01 12:10:00
28	28	fadywaheed	pass123	Regular_User	2026-04-01 12:15:00
29	29	ranaessam	pass123	Regular_User	2026-04-01 12:20:00
30	30	karimatef	pass123	Regular_User	2026-04-01 12:25:00
31	NULL	adminuser	admin123	Admin	2026-04-01 12:30:00
NULL	NULL	NULL	NULL	NULL	NULL

Vehicle_ID	Vehicle_Number	Vehicle_Type	Capacity	Vehicle_Status
1	BUS-101	Bus	50	Active
2	BUS-102	Bus	45	Active
3	BUS-103	Bus	40	Active
4	BUS-104	Bus	50	Active
5	BUS-105	Bus	55	Active
6	METRO-201	Metro	220	Active
7	METRO-202	Metro	220	Active
8	METRO-203	Metro	210	Active
9	TRAIN-301	Train	300	Active
10	TRAIN-302	Train	320	Active
11	TRAIN-303	Train	310	Active
12	TRAIN-304	Train	350	Active
13	BUS-106	Bus	45	Maintenance
14	BUS-107	Bus	50	Active
15	BUS-108	Bus	40	Active
16	METRO-204	Metro	230	Active
17	METRO-205	Metro	220	Maintenance
18	TRAIN-305	Train	330	Active
19	TRAIN-306	Train	340	Active
20	TRAIN-307	Train	360	Active
NULL	NULL	NULL	NULL	NULL

2.8 Vehicle

3.2 SQL Queries

SELECT Operation

This query searches for schedules for a specific route and travel date.

```
SELECT *
FROM Schedule
WHERE Route_ID = 1
AND Travel_Date = '2026-05-01';
```

INSERT Operation

This query inserts a new ticket record.

```
INSERT INTO Ticket
VALUES (31, 1, 1, 30, '2026-04-26 09:00:00',
        17.75, 'Booked');
```

UPDATE Operation

This query decreases the number of available seats after a booking.

```
UPDATE Schedule
SET Available_Seats = Available_Seats - 1
WHERE Schedule_ID = 1;
```

DELETE Operation

This query deletes a ticket record.

```
DELETE FROM Ticket
WHERE Ticket_ID = 32;
```

JOIN Operation 1

This query checks ticket fares by comparing the stored fare with the calculated fare from the route.

```
SELECT
    Ticket.Ticket_ID,
    Route.Route_Name,
    Route.Base_Fare,
    Route.Distance_KM,
    Ticket.Fare_Amount,
    Route.Base_Fare + (Route.Distance_KM * 0.50) AS Correct_Fare
FROM Ticket
JOIN Schedule ON Ticket.Schedule_ID = Schedule.Schedule_ID
JOIN Route ON Schedule.Route_ID = Route.Route_ID;
```

JOIN Operation 2

This query checks whether booked seat numbers are within the vehicle capacity.

```
SELECT
    Ticket.Ticket_ID,
    Ticket.Seat_Number,
    Vehicle.Vehicle_Number,
    Vehicle.Capacity
FROM Ticket
JOIN Schedule ON Ticket.Schedule_ID = Schedule.Schedule_ID
JOIN Vehicle ON Schedule.Vehicle_ID = Vehicle.Vehicle_ID;
```

Aggregate Functions

The following queries count the total number of tickets and calculate total revenue.

```
SELECT COUNT(*) AS Total_Tickets
FROM Ticket;

SELECT SUM(Payment_Amount) AS Total_Revenue
FROM Payment;
```

Subquery

This query displays tickets with fare amounts greater than the average fare.

```
SELECT *
FROM Ticket
WHERE Fare_Amount > (
    SELECT AVG(Fare_Amount)
    FROM Ticket
);
```

Ticket_ID	Passenger_ID	Schedule_ID	Seat_Number	Booking_Date	Fare_Amount	Ticket_Status
8	8	8	214	2026-04-25 09:35:00	130.00	Booked
16	16	16	301	2026-04-25 10:15:00	142.50	Booked
17	17	17	345	2026-04-25 10:20:00	207.50	Booked
18	18	18	266	2026-04-25 10:25:00	255.00	Booked
19	19	19	115	2026-04-25 10:30:00	360.00	Booked
22	22	22	120	2026-04-25 10:45:00	130.00	Booked
25	25	25	346	2026-04-25 11:00:00	360.00	Booked
28	28	8	108	2026-04-25 11:15:00	130.00	Booked
29	29	19	215	2026-04-25 11:20:00	360.00	Booked
30	30	25	25	2026-04-25 11:25:00	360.00	Booked
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 3: Sample SQL Query Result

3.3 Stored Procedures, Triggers, Transactions

A stored procedure was used to calculate the final fare for a route. The calculation is based on the base fare and the route distance.

```
DROP PROCEDURE IF EXISTS CalculateFare;

DELIMITER //

CREATE PROCEDURE CalculateFare(IN routeId INT)
BEGIN
    SELECT
        Route_ID,
        Route_Name,
        Base_Fare,
        Distance_KM,
        Base_Fare + (Distance_KM * 0.50) AS Final_Fare
    FROM Route
    WHERE Route_ID = routeId;
END //

DELIMITER ;

CALL CalculateFare(1);
```

A transaction was also used for ticket booking. This is important because booking a ticket involves more than one step. The system must insert the ticket, insert the payment, and update the available seats. These steps should be treated as one unit.

```
START TRANSACTION;

INSERT INTO Ticket
VALUES (31, 1, 1, 30, '2026-04-26 09:00:00', 17.75, 'Booked');

INSERT INTO Payment
VALUES (31, 31, 'Cash', 17.75,
        '2026-04-26 09:01:00', 'Paid');
```

```
UPDATE Schedule
SET Available_Seats = Available_Seats - 1
WHERE Schedule_ID = 1;

COMMIT;
```

```
DROP PROCEDURE IF EXISTS CalculateFare;


DELIMITER //

CREATE PROCEDURE CalculateFare(IN routeId INT)
BEGIN
    SELECT
        Route_ID,
        Route_Name,
        Base_Fare,
        Distance_KM,
        Base_Fare + (Distance_KM * 0.50) AS Final_Fare
    FROM Route
    WHERE Route_ID = routeId;
END //

DELIMITER ;


CALL CalculateFare(1);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Route_ID	Route_Name	Base_Fare	Distance_KM	Final_Fare
1	Cairo to Giza	5.00	25.50	17.7500

18	14:53:15	DROP PROCEDURE IF EXISTS CalculateFare	0 row(s) affected
19	14:53:15	CREATE PROCEDURE CalculateFare(IN routeId INT) BEGIN SELECT Route_ID, Route_Name, Base_Fare, Distance_KM, B...	0 row(s) affected
20	14:53:15	CALL CalculateFare(1)	1 row(s) returned

Figure 4.1 : Stored Procedure

```
UPDATE Schedule
SET Available_Seats = Available_Seats - 1
WHERE Schedule_ID BETWEEN 1 AND 25;

UPDATE Schedule
SET Available_Seats = Available_Seats - 1
WHERE Schedule_ID IN (1, 2, 8, 19, 25);

COMMIT;
```

Schedule_ID	Route_ID	Vehicle_ID	Departure_Time	Arrival_Time	Travel_Date	Available_Seats	Schedule_Status
1	1	1	08:00:00	09:00:00	2026-05-01	45	Scheduled
2	2	2	09:00:00	10:00:00	2026-05-01	41	Scheduled
3	3	6	10:00:00	10:45:00	2026-05-01	218	Scheduled
4	4	7	11:00:00	12:00:00	2026-05-01	218	Scheduled
5	5	8	12:00:00	12:30:00	2026-05-01	208	Scheduled
6	6	3	13:00:00	14:30:00	2026-05-01	38	Scheduled
7	7	4	14:00:00	15:15:00	2026-05-01	48	Scheduled
8	8	9	07:00:00	10:30:00	2026-05-02	296	Scheduled
9	9	10	08:00:00	09:45:00	2026-05-02	318	Scheduled
10	10	11	09:00:00	11:15:00	2026-05-02	308	Scheduled
11	11	12	10:00:00	11:30:00	2026-05-02	348	Scheduled
12	12	14	11:00:00	13:00:00	2026-05-02	48	Scheduled
13	13	15	12:00:00	14:15:00	2026-05-02	38	Scheduled
14	14	5	13:00:00	15:00:00	2026-05-02	53	Scheduled
15	15	18	14:00:00	16:20:00	2026-05-02	328	Scheduled
16	16	19	15:00:00	19:00:00	2026-05-03	338	Scheduled
17	17	20	16:00:00	22:00:00	2026-05-03	358	Scheduled
18	18	9	17:00:00	23:30:00	2026-05-03	298	Scheduled
19	19	10	18:00:00	05:00:00	2026-05-03	316	Scheduled
20	20	11	19:00:00	21:00:00	2026-05-03	308	Scheduled
21	1	2	16:00:00	17:00:00	2026-05-04	43	Scheduled
22	8	12	06:30:00	10:00:00	2026-05-04	348	Scheduled
23	9	18	07:30:00	09:15:00	2026-05-04	328	Scheduled
24	12	19	08:30:00	10:45:00	2026-05-04	338	Scheduled
25	19	20	20:00:00	07:00:00	2026-05-04	356	Scheduled
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

21	14:54:20	UPDATE Schedule SET Available_Seats = Available_Seats - 1 WHERE Schedule_ID BETWEEN 1 AND 25	25 row(s) affected Rows matched: 25 Changed: 25 Warnings: 0
22	14:54:20	UPDATE Schedule SET Available_Seats = Available_Seats - 1 WHERE Schedule_ID IN (1, 2, 8, 19, 25)	5 row(s) affected Rows matched: 5 Changed: 5 Warnings: 0
23	14:54:20	COMMIT	0 row(s) affected

Figure 4.2 : Transaction Execution

4 Query Optimization

Query optimization was applied using indexing. Since users will often search for schedules by route, travel date, and departure time, an index was created on these columns in the Schedule table.

```
CREATE INDEX schedule_search_index
ON Schedule (Route_ID, Travel_Date, Departure_Time);
```

This index helps the database find matching schedule records faster. Without an index, the database may need to scan the full Schedule table to find matching rows. With the index, the DBMS can locate the required rows more efficiently.
The following query was used to test the index:

```
SELECT *
FROM Schedule
WHERE Route_ID = 1
AND Travel_Date = '2026-05-01';
```

The EXPLAIN command can be used to view the query execution plan:

```
EXPLAIN
SELECT *
FROM Schedule
WHERE Route_ID = 1
AND Travel_Date = '2026-05-01';
```

```
ON Schedule (Route_ID, Travel_Date, Departure_Time);

SELECT *
FROM Schedule
WHERE Route_ID = 1
AND Travel_Date = '2026-05-01';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Schedule_ID	Route_ID	Vehicle_ID	Departure_Time	Arrival_Time	Travel_Date	Available_Seats	Schedule_Status
1	1	1	08:00:00	09:00:00	2026-05-01	45	Scheduled
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
EXPLAIN
SELECT *
FROM Schedule
WHERE Route_ID = 1
AND Travel_Date = '2026-05-01';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Schedule	NULL	ref	schedule_search_index	schedule_search_index	9	const,const	1	100.00	NULL

Figure 5: Query Execution Plan

The selected index is useful because the most common schedule search depends on Route_ID and Travel_Date. Departure_Time was also included because users may want to sort or filter trips by departure time.

5 Data Integrity and Security

5.1 Constraints and Data Validation

Data integrity was maintained using primary keys and foreign keys. Primary keys ensure that each record is uniquely identified. Foreign keys ensure that a record in one table refers to an existing record in another table.

Examples of primary keys in the system include Passenger ID, Station ID, Vehicle ID, Route ID, Schedule ID, Ticket ID, and Payment ID.

Examples of foreign keys include:

- User_Account.Passenger_ID references Passenger.Passenger_ID.
- Route.Start_Station_ID references Station.Station_ID.
- Route.End_Station_ID references Station.Station_ID.
- Route_Stop.Route_ID references Route.Route_ID.
- Route_Stop.Station_ID references Station.Station_ID.
- Schedule.Route_ID references Route.Route_ID.
- Schedule.Vehicle_ID references Vehicle.Vehicle_ID.

- Ticket.Passenger_ID references Passenger.Passenger_ID.
- Ticket.Schedule_ID references Schedule.Schedule_ID.
- Payment.Ticket_ID references Ticket.Ticket_ID.

Additional constraints can be added to improve validation. For example:

```
ALTER TABLE Passenger
ADD CONSTRAINT unique_passenger_email UNIQUE (Email);

ALTER TABLE User_Account
ADD CONSTRAINT unique_username UNIQUE (Username);

ALTER TABLE Vehicle
ADD CONSTRAINT unique_vehicle_number UNIQUE (Vehicle_Number);
```

Check constraints can also be used to prevent invalid values.

```
ALTER TABLE Vehicle
ADD CONSTRAINT check_vehicle_capacity
CHECK (Capacity > 0);

ALTER TABLE Schedule
ADD CONSTRAINT check_available_seats
CHECK (Available_Seats >= 0);

ALTER TABLE Route
ADD CONSTRAINT check_route_distance
CHECK (Distance_KM > 0);
```

These constraints help reduce invalid data. For example, a vehicle should not have zero or negative capacity, and a route should not have a negative distance.

5.2 Access Control and Security Measures

The User_Account table includes the User_Role attribute. This supports two main types of users:

- Admin: manages stations, vehicles, routes, schedules, and system data.
- Regular User: views schedules, books tickets, and makes payments.

The database includes a Password_Hash column to support secure password storage. In the current sample data, simple values were used for testing, but in a real system these values should be stored as hashed passwords.

Recommended security policies include:

- Only administrators should be allowed to insert, update, or delete stations, vehicles, routes, and schedules.

- Regular users should only be able to view schedules and manage their own bookings.
- Payment records should only be accessed by authorized users.
- Database backups should be created regularly.
- User passwords should never be stored as plain text.

6 Results and Discussion

6.1 Testing and Validation

The system was tested by inserting sample data into all main tables. The data inserted included passengers, user accounts, stations, vehicles, routes, route stops, schedules, tickets, and payments.

The database was tested using different SQL operations. SELECT queries were used to retrieve schedules and tickets. INSERT queries were used to add new records. UPDATE queries were used to modify seat availability and passenger data. DELETE queries were used to remove test records. Join queries were used to validate fare calculations and seat numbers. Aggregate queries were used to count tickets and calculate total revenue.

The transaction test was one of the most important parts because ticket booking requires more than one database operation. The transaction made sure that the ticket record, payment record, and seat update are handled together.

```sql
SELECT
    Ticket.Ticket_ID,
    Route.Route_Name,
    Route.Base_Fare,
    Route.Distance_KM,
    Ticket.Fare_Amount,
    Route.Base_Fare + (Route.Distance_KM * 0.50) AS Correct_Fare
FROM Ticket
JOIN Schedule ON Ticket.Schedule_ID = Schedule.Schedule_ID
JOIN Route ON Schedule.Route_ID = Route.Route_ID;
```

Ticket_ID	Route_Name	Base_Fare	Distance_KM	Fare_Amount	Correct_Fare
1	Cairo to Giza	5.00	25.50	17.75	17.7500
26	Cairo to Giza	5.00	25.50	17.75	17.7500
31	Cairo to Giza	5.00	25.50	17.75	17.7500
21	Cairo to Giza	5.00	25.50	17.75	17.7500
2	Cairo to Nasr City	5.00	18.00	14.00	14.0000
27	Cairo to Nasr City	5.00	18.00	14.00	14.0000
3	Cairo to Maadi	5.00	15.00	18.00 2.50	12.5000
4	Maadi to Helwan	5.00	20.00	15.00	15.0000
5	Giza to Dokki	5.00	8.00	9.00	9.0000
6	Giza to October	7.00	30.00	22.00	22.0000
7	Nasr City to New...	7.00	22.00	18.00	18.0000
8	Cairo to Alexandria	20.00	220.00	130.00	130.0000
28	Cairo to Alexandria	20.00	220.00	130.00	130.0000
22	Cairo to Alexandria	20.00	220.00	130.00	130.0000
9	Cairo to Tanta	15.00	95.00	62.50	62.5000
23	Cairo to Tanta	15.00	95.00	62.50	62.5000
10	Cairo to Mansoura	15.00	125.00	77.50	77.5000
11	Cairo to Zagazig	15.00	85.00	57.50	57.5000
12	Cairo to Ismailia	15.00	120.00	75.00	75.0000
24	Cairo to Ismailia	15.00	120.00	75.00	75.0000
13	Cairo to Suez	15.00	135.00	82.50	82.5000
14	Cairo to Fayoum	15.00	100.00	65.00	65.0000
15	Cairo to Beni Suef	15.00	115.00	72.50	72.5000
16	Cairo to Minya	20.00	245.00	142.50	142.5000
17	Cairo to Asyut	20.00	375.00	207.50	207.5000
18	Cairo to Sohag	20.00	470.00	255.00	255.0000
19	Cairo to Luxor	25.00	670.00	360.00	360.0000
29	Cairo to Luxor	25.00	670.00	360.00	360.0000
25	Cairo to Luxor	25.00	670.00	360.00	360.0000
30	Cairo to Luxor	25.00	670.00	360.00	360.0000
20	Alexandria to Ta...	15.00	110.00	70.00	70.0000

Figure 6: Testing and Validation Results

6.2 Challenges and Limitations

One challenge in the project was designing the relationship between routes and stations. A route does not only have a start station and an end station. It may also pass through several intermediate stations. To solve this problem, the Route Stop table was added. This table stores the stations included in a route and their order.

Another challenge was keeping ticket booking consistent. If a ticket is inserted but the available seats are not updated, the system will show incorrect seat availability. This was handled using a transaction.

The current project is limited to the database layer only. It does not include a complete user interface, real online payments, or real-time vehicle location tracking. Also, the trigger for automatic seat updates is optional and should be used carefully to avoid decreasing seats twice.

7 Conclusion

The Public Transport Management System database provides a structured way to manage passengers, stations, vehicles, routes, schedules, tickets, and payments. The database design uses clear relationships between tables and follows normalization rules up to Third Normal Form.

The project includes important database concepts such as primary keys, foreign keys, CRUD operations, joins, aggregate functions, subqueries, stored procedures, transactions, indexing, and data validation. The Route Stop table was used to solve the many-to-many relationship between routes and stations, and transactions were used to make the ticket booking process more reliable.

Future improvements could include building a web or mobile interface, adding real online payment integration, adding real-time vehicle tracking, improving user access control, and using triggers more fully to automate some database operations.

8 References

- Course lecture slides, Fundamentals of Database Systems, Entity-Relationship Model and Relational Database Constraints.
- MySQL Documentation, Oracle Corporation.